

**ipb****INSTITUTO POLITÉCNICO DE BRAGANÇA**
Escola Superior de Tecnologia e Gestão**Engenharia Informática - Arquitetura de Computadores**
Teste Intermédio de 24/04/2018 - [1h30m]

Nota: resolva a questão 1 numa folha, e as questões 2 e 3 noutra folha

1. (Cap 4) Considere o seguinte programa, codificado em assembly MARIE:

```
//Etiqueta    // Diretivas, Instruções e Dados
```

```
ORG 300
```

```
UpdateA,
```

```
Load A
```

```
Add A
```

```
Store A
```

```
Jump PrintA
```

```
UpdateB,
```

```
Load B
```

```
Add B
```

```
Store B
```

```
Jump PrintB
```

```
Finish,
```

```
Halt
```

```
PrintA,
```

```
Load A
```

```
Output
```

```
Jump UpdateB
```

```
PrintB,
```

```
Load B
```

```
Output
```

```
Jump Finish
```

```
A,
```

```
Dec 6
```

```
B,
```

```
Dec 7
```

Códigos de operação (HEX) e mnemónicas do assembly MARIE

(1) Load X	(2) Store X	(3) Add X	(4) Subt X
(5) Input	(6) Output	(7) Halt	(9) Jump X
(0) JnS X	(A) Clear	(B) AddI X	(C) JumpI X
(8) Skipcond X , em que: (X == 000) => (AC<0 ?)			
(X == 400) => (AC==0?)			
(X == 800) => (AC>0 ?)			

Descrições RTL da execução de Load X, Store X, Add X e Jump X:

Load X	MAR ← X; MBR ← Memory[MAR]; AC ← MBR
Store X	MAR ← X; MBR ← AC; Memory[MAR] ← MBR
Add X	MAR ← X; MBR ← Memory[MAR]; AC = AC + MBR
Jump X	PC ← IR[11-0] (PC ← 12 bits menos significativos de IR)

Assemble o programa anterior e apresente na sua folha de resolução:

a) a tabela de símbolos que resulta da primeira passagem;

b) o resultado final da assemblagem, em formato hexadecimal, que resulta da segunda passagem;

c) os valores dos registos PC, IR, MAR, MBR e AC após a execução **c1)** da 1ª instrução (Load A), **c2)** da 2ª instrução (Add A), **c3)** da 3ª instrução (Store A), e da **c4)** da 4ª instrução (Jump PrintA)

2. (Cap 4)

Considere o seguinte código C de um programa que lê dois números inteiros (aa e bb) e determina o quociente (qq) da divisão (inteira) do primeiro pelo segundo, recorrendo a uma função (Quociente).

```
int Quociente (int a, int b) {
    int r, q;
    r = a; q = 0;
    while ( r - b + 1 > 0) {
        r = r - b;
        q = q + 1;
    }
    return(q);
}
```

```
int main() {
    int aa, bb, qq;
    scanf("%d %d", &aa, &bb);
    qq = Quociente(aa, bb);
    printf("%d\n", qq);
}
```

Traduza, para assembly MARIE, o código C: **a)** da função main, **b)** da função Quociente.

3. (Cap 4)

Considere o seguinte código C de um programa que calcula e apresenta, para cada elemento de um array (aa), o quociente da divisão (inteira) desse elemento por 2. O programa em causa assume a disponibilidade da função `Quociente`, previamente introduzida e definida na questão 2.

```
int Quociente (int a, int b) { . . . }
int main() {
    int aa[5]={1,2,3,4,5}, ii;
    for (ii=0; ii<5; ii++)
        printf("%d\n", Quociente(aa[ii],2));
}
```

O código abaixo é uma implementação em MARIE incompleta da função `main` anterior. Apresente, na sua folha de resolução, os **BLOCOS** de código MARIE em falta, de forma a respeitar o código C anterior.

```
ORG 100

/ BLOCO 1: testa se ArrayCellIndex (ii) é menor que 5
Loop,
____
____
____
Jump      EndLoop

/ BLOCO 2: coloca em ArrayCellAddress o endereço de aa[ii]
____
____
____

/ BLOCO 3: coloca em AC o conteúdo de aa[ii]
____
____

/ BLOCO 4: printf("%d\n", Quociente(aa[ii],2));
Store     a
Load      Two
Store     b
JnS       Quociente
Load      q
Output

/ BLOCO 5: incrementa o índice de acesso ao array (ii++)
____
____
____

Jump      Loop
EndLoop,  Halt

Dec       10          / aa[0]
Dec       20          / aa[1]
Dec       30          / aa[2]
Dec       40          / aa[3]
Dec       50          / aa[4]
ArrayBaseAddress, Hex
ArrayCellAddress, Hex 000 / BLOCO 6: endereço de aa[0]
ArrayCellIndex,  Dec  0  / endereço de aa[ii]
One,         Dec  1    / índice ii
Two,         Dec  2    / constante 1
Five,        Dec  5    / constante 2
Five,        Dec  5    / constante 5
```